Nourish SLM: Technical Report

KM Sreenidhi · Woxsen University · Built for Veltris · Oct 13, 2026

Summary

Nourish SLM is a personalized recipe assistant that turns the ingredients a user has into a complete recipe that respects their diet. It runs on an open-weight small language model, Qwen3-4B, fine-tuned with LoRA on Food.com recipes. Every answer passes through a deterministic dietary validator before it reaches the user.

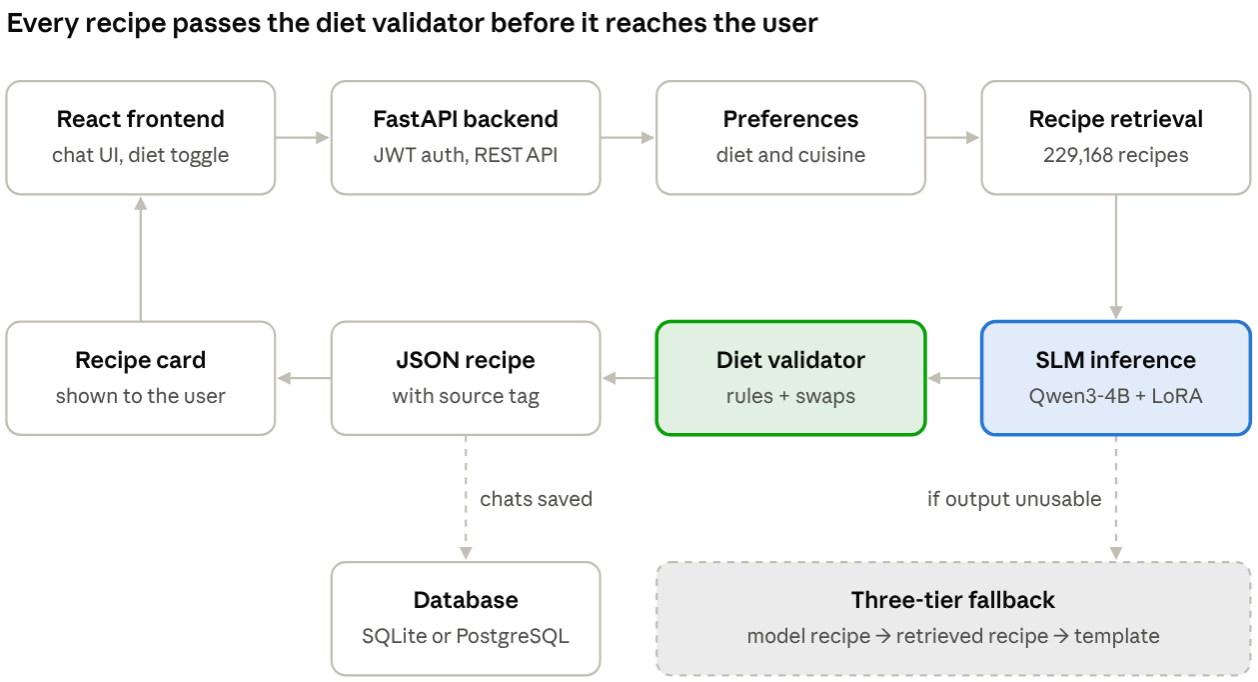
On held-out recipes, fine-tuning cut test loss by 62% (2.895 to 1.106) and raised the share of complete, well-formed recipes from 66.7% to 97.0%. Dietary compliance stayed at 100%. The system runs fully on the owner's hardware: a MacBook (Apple Silicon), a CUDA GPU server, or Docker, with no external LLM API.

What the product does

A user signs in, picks a dietary preference, and describes what is in their kitchen in plain language, for example "I have chickpeas, onions and tomatoes, make me something Indian." Nourish replies with a structured recipe card.

- **Diet preferences:** Vegan, Vegetarian, Eggitarian, Non-vegetarian and No preference. The preference can be switched from the header at any time, without resetting the chat or retraining.
- **Cuisines:** Indian, Italian and Continental, or any cuisine.
- **Recipe card:** title, diet badge, cuisine, prep and cook time, servings, checkable ingredients, numbered steps and a "Dietary validated" badge.
- **Account features:** chat history, saved recipes, favorites, settings, and light and dark themes.

System architecture



request flow · 8 components, 1 fallback path

A request flows from the React app through the FastAPI backend, which applies the user's diet and cuisine, retrieves similar recipes and asks the fine-tuned model for a JSON recipe. The validator checks it before the card is shown. If the model's output can't be used, the backend falls back to a retrieved dataset recipe, then to a template, so every response reports its source and none is empty.

Data pipeline

The model learns from the Food.com recipe dataset (`RAW_recipes.csv`): 231,637 recipes with name, ingredients, steps, tags, nutrition and cooking time.

1. **Inspect and clean** (`inspect_dataset.py`, `preprocess_dataset.py`): drop invalid records, de-duplicate and normalize ingredient names. This leaves 229,168 usable recipes.
2. **Label by diet** (`classify_diet.py`): every recipe is tagged Non-vegetarian (41.7%), Eggitarian (21.1%), Vegetarian (20.8%) or Vegan (16.4%) using the same rules the validator applies.
3. **Build instructions** (`create_instruction_dataset.py`): recipes become prompt and response pairs from 5 templates, such as ingredients to recipe, diet-constrained recipes and cuisine-specific dishes. Responses are JSON recipes.

4. **Split** (`split_dataset.py`): 80/10/10 train, validation and test files. The fine-tuning run used 6,000 training and 200 validation examples, with the test split held out for evaluation.

Model and fine-tuning

The base model is Qwen/Qwen3-4B, an open-weight 4-billion-parameter language model. Its weights stay frozen; LoRA (low-rank adaptation) trains a small update on top, under 1% of the model's parameters, saved as a 132 MB adapter in `notebooks/models/qwen3-4b-recipe-lora` .

Setting	Value
Method	LoRA via Hugging Face PEFT, TRL and Transformers
Rank / alpha / dropout	r = 16, α = 32, 0.05
Target modules	q, k, v, o, gate, up and down projections
Training data	6,000 examples, 1 epoch, 375 optimizer steps
Batch	1 per device × 16 gradient accumulation = 16
Learning rate / max length	2e-4 / 640 tokens
Hardware / time	NVIDIA RTX 4070, 55.4 minutes
Loss	train 1.191, best validation 1.145

The full training workflow is in `notebooks/qwen3_4b_finetuningFinal.ipynb` and `scripts/train_qwen3_4b_lora.py` .

Dietary safety layer

The model suggests a recipe, but a rule-based validator decides what the user sees. Each ingredient is mapped to a group (meat, poultry, fish, seafood, egg, dairy or plant) and checked against the user's preference.

Preference	Meat, poultry, fish, seafood	Egg	Dairy	Plant
Vegan	Not allowed	Not allowed	Not allowed	Allowed
Vegetarian	Not allowed	Not allowed	Allowed	Allowed
Eggitarian	Not allowed	Allowed	Allowed	Allowed

Preference	Meat, poultry, fish, seafood	Egg	Dairy	Plant
Non-vegetarian	Allowed	Allowed	Allowed	Allowed

When an ingredient breaks the rule, it is swapped for a compliant one and the steps are rewritten to match; for example, ghee in a vegan recipe becomes olive oil. Matching is whole-word, so plant items like almond milk, coconut cream, peanut butter and eggplant are not flagged. The validator passed 50 of 50 test cases at about 0.1 ms per check.

Backend and frontend

Backend: Python FastAPI app (`backend/app/main.py`) served by Uvicorn on port 8000. Users and passwords use JWT tokens (python-jose) and bcrypt hashing. Data lives in SQLite by default (`food_slm.db`) or PostgreSQL through SQLAlchemy. Inference uses PyTorch with Transformers and PEFT, loading the base model and adapter once at startup.

Endpoint	Purpose
`POST /api/auth/register`, `/api/auth/login`	Create account, sign in, return token
`GET /api/user/profile`, `PUT /api/user/preferences`	Read profile, change diet
`POST /api/chat`	Generate a validated recipe
`GET /api/chat/history`, `/api/chat/{id}`	List and open past chats
`/api/recipes/save`, `/api/favorites`	Saved recipes and favorites
`GET /api/model-status`, `/health`	Model loaded, device, health check

Frontend: React 18 with TypeScript, built with Vite into `frontend/dist` and served on port 5173. It has login, registration, onboarding, the chat dashboard, saved recipes and settings pages, and calls the API with Axios.

Evaluation

The fine-tuned model beats the base Qwen3-4B on every quality measure while keeping diet compliance at 100%. Both were run on the same prompts and the same held-out test split.

Metric	Base Qwen3-4B	Fine-tuned (LoRA)
Test loss	2.895	1.106 (62% lower)
Perplexity	18.1	3.0
Format completeness (usable JSON recipe)	66.7%	97.0%
Dietary compliance	100%	100%

The validator was tested separately on 50 hand-written ingredient lists (15 vegan, 15 vegetarian, 10 eggitarian, 10 non-vegetarian): 50 passed, 0 conflicts missed. Reports are in `data/evaluation/` and `notebooks/results/`.

Deployment

Nourish SLM deploys in three ways, all self-hosted. Each needs about 8 GB for the base model weights, which download once from Hugging Face, plus the 132 MB adapter.

Option	Hardware	How it runs
Local Mac	Apple Silicon, 16 GB RAM (tested on M3)	`run_on_mac.command` or two terminals: Uvicorn backend on 8000, static frontend on 5173; PyTorch MPS
GPU server	NVIDIA CUDA GPU, 12 GB+ VRAM (trained on RTX 4070)	Same backend on CUDA, frontend served from `frontend/dist` behind a web server
Docker	Any host with Docker	`docker-compose up --build` starts the backend and PostgreSQL

Running locally:

```
source .venv-mac/bin/activate
export BASE_MODEL_NAME="Qwen/Qwen3-4B"
export FINAL_ADAPTER_DIR="$PWD/notebooks/models/qwen3-4b-recipe-lora"
export SLM_GENERATE=true SLM_PRELOAD=true PYTORCH_ENABLE_MPS_FALLBACK=1
python -m uvicorn backend.app.main:app --host 127.0.0.1 --port 8000
# second terminal:
python3 -m http.server 5173 --bind 127.0.0.1 --directory frontend/dist
```

Configuration (`.env`): `SECRET_KEY` for JWT signing (required in production), `DATABASE_URL` for PostgreSQL, `CORS_ORIGINS` , `BASE_MODEL_NAME` , `SLM_GENERATE` (set false for a fast retrieval-only mode) and `SLM_MAX_NEW_TOKENS` . For production, put the API behind HTTPS and set a fixed `SECRET_KEY` so sign-ins survive restarts.

Deployment steps

Follow one of the three paths below. Before any of them, add the two packages the backend needs but `requirements.txt` does not list yet:

```
echo "email-validator" >> requirements.txt
echo "python-multipart" >> requirements.txt
```

A. Local Mac (Apple Silicon)

1. Install Python 3.12 and Node.js 18 or later.
2. Create the environment and install packages:

   ```
   cd ~/Downloads/"Food SLM copy 2"
   python3 -m venv .venv-mac && source .venv-mac/bin/activate
   grep -v -E "psycopg2|asyncpg" requirements.txt > /tmp/req.txt
   pip install -r /tmp/req.txt accelerate huggingface_hub
   ```

3. Download the base model once (about 8 GB):

   ```
   python -c "from huggingface_hub import snapshot_download;
   snapshot_download('Qwen/Qwen3-4B')"
   ```

4. Create the config and a secret key:

   ```
   cp .env.example .env
   echo "SECRET_KEY=$(python3 -c 'import secrets;
   print(secrets.token_urlsafe(48))')" >> .env
   ```

5. Start the backend and leave it running:

   ```
   export BASE_MODEL_NAME="Qwen/Qwen3-4B"
   export FINAL_ADAPTER_DIR="$PWD/notebooks/models/qwen3-4b-recipe-lora"
   export SLM_GENERATE=true SLM_PRELOAD=true PYTORCH_ENABLE_MPS_FALLBACK=1
   python -m uvicorn backend.app.main:app --host 127.0.0.1 --port 8000
   ```

6. In a second terminal, serve the frontend:

```
cd ~/Downloads/"Food SLM copy 2"
python3 -m http.server 5173 --bind 127.0.0.1 --directory frontend/dist
```

7. Open http://localhost:5173. To stop, press Ctrl+C in both terminals.

B. Docker (backend + PostgreSQL)

1. Install Docker. On a Mac, containers cannot use the Apple GPU, so the model runs on CPU and is slow; use a Linux host with an NVIDIA GPU for real use.
2. Point the container at the trained adapter: in `docker-compose.yml`, under `backend` → `environment`, add

```
FINAL_ADAPTER_DIR: /app/notebooks/models/qwen3-4b-recipe-lora
```

3. Set secrets in `.env`:

```
SECRET_KEY=<a long random string>
POSTGRES_PASSWORD=<a strong password>
```

4. Build and start:

```
docker-compose up --build -d
curl http://localhost:8000/health
```

5. Serve `frontend/dist` from any static web server (step C5 shows Nginx).

C. Linux GPU server (production)

1. Install the NVIDIA driver, Python 3.12, PostgreSQL and Nginx. Install a CUDA build of PyTorch, then the rest of `requirements.txt`.
2. Copy the project to the server, for example to `/opt/nourish`, and download the base model as in A3.
3. Point the frontend at the server and rebuild it. In `frontend/src/services/api.ts`, change `API_BASE_URL` from `http://localhost:8000/api` to `/api`, then:

```
cd frontend && npm install && npm run build
```

4. Set `.env`: `SECRET_KEY`, `DATABASE_URL=postgresql://fooduser:<password>@localhost:5432/foodslmdb`, `CORS_ORIGINS=https://your-domain`, `BASE_MODEL_NAME=Qwen/Qwen3-4B` and `FINAL_ADAPTER_DIR=/opt/nourish/notebooks/models/qwen3-4b-recipe-lora`.
5. Run the backend as a service with a single worker, since each worker loads its own copy of the model:

```
uvicorn backend.app.main:app --host 127.0.0.1 --port 8000 --workers 1
```

Wrap this in a systemd unit so it restarts on failure and on boot.

6. Configure Nginx to serve the site and proxy the API:

```
server {
  server_name your-domain;
  root /opt/nourish/frontend/dist;
  location / { try_files $uri /index.html; }
  location /api/ { proxy_pass http://127.0.0.1:8000; proxy_read_timeout
120s; }
}
```

7. Add HTTPS with Certbot (`certbot --nginx -d your-domain`), then check https://your-domain and `/api/model-status`, which should report the model and device.

Limitations and next steps

- **Keyword-based diet checks:** hidden animal products such as Worcestershire sauce or naan can be missed. The validator is not a substitute for allergen or medical advice.
- **Short training run:** one epoch on 6,000 examples. Training on more of the 229,168 recipes should further improve recipe quality.
- **Latency:** generation with the 4B model on a laptop takes several seconds per recipe; a quantized model or a GPU server would speed this up.
- **Cuisine coverage:** the interface offers Indian, Italian and Continental today.